UX Heuristics and Laws: A Deep Dive

Level 2 of the fomodoro UX reading series. One level deeper on Nielsen's ten heuristics and the "laws of UX", with examples from fomodoro. Prepared November 2026.

Level 2 of the UX series. Prerequisite: *ux-primer.pdf* (done). Next: *usability-fundamentals.pdf*. Time: ~45 minutes.
Sections: 1. Ground rules (3 min) · 2. Where the heuristics came from (3 min) · 3. The ten heuristics in depth (18 min) · 4. Running a heuristic evaluation solo (4 min) · 5. The laws of UX in depth (15 min) · 6. Review checklist (2 min) · 7. Where this goes next.

Evidence labels (same scheme as the primer).

STRONG Replicated empirical research, or a formal standard (such as WCAG).

MODERATE Some empirical support, but limited, dated, proprietary, or contested in scope.

HEURISTIC Expert or practitioner guidance and observation; not experimentally established. Official platform guidance (Apple HIG) gets this label too: it is the authority on that platform's conventions, but it is not itself evidence.

WEAK Tested empirically, but current evidence does not support the popular claim.

As before: a Strong lab finding does not make our app-specific use of it Strong. Examples in green are fomodoro. Bracketed numbers like [1] point to the sources at the end. Sentences starting "We recommend" are our judgment, not findings. Real-product examples are described from general knowledge to illustrate a point, not as research findings.

1. Ground rules for this document

The authority order

Heuristics and laws are the vocabulary of design review, but they are not the top of the decision chain. For UI and UX decisions in fomodoro, the order of authority is:

1. **WCAG 2.2 Level AA** STRONG [10]. If a design fails an AA success criterion, it is wrong, however elegant it looks.
2. **Apple's Human Interface Guidelines** HEURISTIC [12]. Our users are on iPhone and Mac; HIG conventions are what "normal" feels like there.
3. **Strong-evidence research**, such as Fitts's law or the limits of working memory.
4. **Heuristics**, including Nielsen's ten and the practitioner "laws".

When a heuristic "aligns with" WCAG, it is backed by something stronger; when it "conflicts", the higher authority wins and the heuristic must be satisfied some other way. In practice the real conflicts are few and fairly predictable: minimalism against visible labels and contrast, efficiency shortcuts against keyboard accessibility, and screen-edge targets against iOS system gestures. Each one is flagged where it comes up.

Where we checked a primer claim and found it needs refining, a **Correction** box says so.

2. Where the ten heuristics came from HEURISTIC

The history matters because it tells you what kind of thing the heuristics are. In 1990 Rolf Molich and Jakob Nielsen published a set of nine dialogue-design principles [2] and, in a companion CHI paper, introduced *heuristic evaluation*: having evaluators inspect an interface against such principles instead of running a user study [1]. Nielsen's 2024 retrospective dates the first list to a course the two taught in 1989, and describes it as produced by discussion rather than systematic research [5].

The 1994 revision is the one everyone uses, and its method is often summarized loosely (including, slightly, in our primer). Here is what the CHI '94 paper actually did [3]. Nielsen gathered seven published sets of usability principles, 101 principles in all, and a database of 249 usability problems from 11 earlier projects (7 evaluated by heuristic evaluation, 4 by user testing). Each of the 101 principles was rated for how well it explained each of the 249 problems. He then ran a factor analysis on those ratings, plus a second analysis that picked the principles giving the broadest coverage of the problems. The factor analysis did *not* produce a tidy short list: it took about 25 factors to account for 62% of the variance, which is itself an important finding (no small set of rules explains all usability problems). The paper's abstract names **nine** heuristics; the published list of ten adds *help and documentation*, and that ten-item list is the one NN/g has carried, essentially unchanged, since 1994 [4, 5]. Nielsen himself has called the number ten somewhat arbitrary [5].

> **Correction to the primer.** The primer says the ten were "derived from a factor analysis of 249 usability problems." Closer to the paper: 101 existing principles were rated against 249 problems from 11 projects; factor and coverage analyses guided a synthesis of nine; the tenth was added in the published list.

Two consequences. The heuristics are *explanatory categories* distilled from past problems: good at naming what is wrong, weaker at predicting it. And they predate smartphones and accessibility law, so WCAG and the HIG are what make them concrete for an iPhone PWA in 2026.

3. The ten heuristics in depth

Each heuristic below is labelled HEURISTIC: expert-derived and widely used, but not experimentally established as a rule. The WCAG references are to success criteria (SC) in WCAG 2.2 [10, 11]; level A or AA is noted.

H1. Visibility of system status HEURISTIC

Recap. The system should always tell people what is going on, through timely feedback.

What it really means. Three things, not one: *state* (what mode am I in, what is selected, am I signed in), *progress* (how far along is this, how long is left), and *feedback* (did my action register). "Timely" is part of the definition: feedback that arrives a second late is a different, worse experience than feedback at 100

ms (see the Doherty threshold in Section 5). It also covers status the user did not ask about but needs, such as "you are offline" or "this is a read-only copy".

Common misreadings. (1) "Show everything." Visibility is about the status relevant to the user's current goal, not a dashboard of internals; it is balanced against H8. (2) "A spinner counts." An indefinite spinner says only "something is happening"; when progress is measurable, show progress. (3) "Visible means visual." Status must also reach people who cannot see the screen, which is where WCAG comes in.

Real products.

- *Good:* ride-hailing apps that show the car moving on the map with a live arrival estimate, turning an anxious wait into a watched process.
- *Bad:* checkout buttons that give no sign of having been pressed, so people press again and risk duplicate orders; or file-copy dialogs whose time estimate swings wildly from minutes to hours.

Review question. Freeze this screen at a random moment: could someone glancing at it say what state the system is in, what just happened, and what happens next?

WCAG and HIG. Aligns with SC 4.1.3 Status Messages (AA): status changes must be announced to assistive technology without moving focus. SC 2.4.7 Focus Visible (AA) is visibility of *keyboard* status. SC 2.2.2 Pause, Stop, Hide (A) is the counterweight: auto-updating content shown alongside other content needs a way to pause it, unless it is essential. The HIG asks for feedback that is clear but not excessive.

In fomodoro. The timer screen is almost entirely H1: draining ring, large digits, block name, cycle dots ("2 of 4"), and "Next: short break, 5 min". The less obvious cases are where the work is. Handoff: a device showing someone else's countdown must say it is read-only and where the timer is running ("Running on iPhone"), not just show digits that look identical to a live timer. Settings: because changes apply from the next block, the settings screen should say so at the moment of change ("Applies from your next block"), or the user will think the change failed. Signed-out mode: a small "Using default times" note explains why the steppers are absent. For screen readers, announce block changes ("Short break started") through a polite live region, but never the per-second tick, which would make the app unusable with VoiceOver. Our countdown starts only on Start and can be paused, which also satisfies 2.2.2.

H2. Match between the system and the real world HEURISTIC

Recap. Speak the user's language, with familiar words, concepts, and ordering, not system-oriented terms.

What it really means. The interface should follow the user's mental model, not the database schema. That covers vocabulary, but also order (things appear in the sequence the user thinks of them), units (minutes, not seconds), and metaphors that carry the right expectations. "Real world" means *the user's* world: for a programmer, "commit" is natural language; for most people it is not.

Common misreadings. (1) "Use skeuomorphism." Matching the real world is about concepts, not leather textures. (2) "Avoid all jargon." Domain terms the user already knows are the match: a Pomodoro app may say "focus block" because its users chose a Pomodoro app.

Real products.

- *Good:* the desktop Trash or Bin: deleted files go somewhere you can retrieve them from until you empty it, exactly as the metaphor promises.
- *Bad:* operating-system error dialogs that show only a hexadecimal error code, or web apps that surface raw parser errors to people who never wrote any JSON.

Review question. Read every label and message aloud to someone who has never seen the code. Which words would they have to ask about?

WCAG and HIG. Aligns with SC 2.4.6 Headings and Labels (AA): labels must describe their purpose. SC 3.1.5 Reading Level is AAA (beyond our target, but a useful aspiration). SC 1.3.3 Sensory Characteristics (A) bans instructions that rely only on shape or position ("tap the round button"). The HIG's writing guidance asks for plain, direct, friendly language.

In fomodoro. "Focus", "Short break", "Long break", "Take over here?" are user words. Watch the security flows, where developer vocabulary leaks in: say "authenticator app" and "6-digit code", not "TOTP" or "OTP"; say "Sign in on this device", not "Create session". Units: steppers show minutes. Order: settings list durations in the order you live them (focus, short break, long break, long break every N).

H3. User control and freedom HEURISTIC

Recap. People make mistakes; give them a clearly marked way out, with undo and redo.

What it really means. Users should feel they are driving. That means exits from any state they wandered into (cancel, back, close), reversal of actions they regret (undo), and no surprises where the system acts on its own in a way they cannot stop. The deeper idea is that easy reversal makes exploration safe, and exploration is how people learn an interface.

Common misreadings. (1) "Add a confirmation dialog." A confirmation is a speed bump, not an exit; people habituate and click through. Undo is usually better for reversible actions; confirmation is for the truly irreversible. (2) "More options means more freedom." Control is about escape and reversal, not about exposing every setting (that is H7, and H8 pushes back).

Real products.

- *Good:* Gmail's "Undo send", which holds a sent message for a few seconds; it turns an irreversible act into a reversible one.
- *Bad:* subscription flows that take one tap to join and many screens, a phone call, or a chat to leave. Regulators now treat the worst of these as deceptive patterns (primer, Section 7).

Review question. From every state on this screen, how does the user get out, and what is the cost of each action they might regret?

WCAG and HIG. Aligns with SC 2.5.2 Pointer Cancellation (A): actions should fire on release, so sliding a finger off a button cancels the tap. SC 3.2.2 On Input (A): changing a setting must not trigger an unexpected change of context. SC 1.4.13 Content on Hover or Focus (AA): popovers must be dismissible. SC 2.2.1 Timing Adjustable (A) is about time limits imposed on the user; a Pomodoro timer is a tool the user chose to run, not a limit on their interaction, but the spirit (the user controls time) is the same. The HIG asks that people can always cancel, and supports standard undo.

In fomodoro. Pause/Resume, Skip, and Reset are the exits. "Take over here?" asks before moving the timer, so opening the app on a Mac never silently steals the timer from the iPhone. One design point the primer glossed over: it said Reset asks nothing because it is cheap to redo. Redoing is cheap; the *lost elapsed time* is not. Resetting 20 minutes into a focus block discards 20 minutes of progress. We recommend no confirmation dialog, but a brief "Block reset. Undo" message that restores the previous state. That keeps Reset fast and makes it reversible. The same pattern suits Skip.

H4. Consistency and standards HEURISTIC

Recap. Users should not have to wonder whether different words, situations, or actions mean the same thing; follow platform conventions.

What it really means. Two layers. *Internal* consistency: within the app, one word for one concept, one place for one control, one look for one kind of element. *External* consistency: behave like the platform and like other apps people use (Jakob's law, Section 5). External usually wins when they conflict, because users spend far more time elsewhere.

Common misreadings. (1) "Everything must look the same." Things that behave differently should look different; false consistency (a destructive button styled like a harmless one) is a bug. (2) "Our brand is the standard." Brand lives in color, type, and voice, not in reinventing a switch. (3) "Consistency means identical across iPhone and Mac." It means each platform's conventions: right-click menus and keyboard shortcuts on Mac, swipe and long-press on iPhone.

Real products.

- *Good:* Command-Z for undo and Command-comma for settings across nearly every Mac app; people try them without thinking.
- *Bad:* apps that put the same action under different names in different places ("Remove", "Delete", "Discard") or move the primary button between screens, so muscle memory sends taps to the wrong control.

Review question. Is every repeated concept named and placed the same way everywhere, and does every control behave the way it does in the platform's own apps?

WCAG and HIG. Strong alignment. SC 3.2.3 Consistent Navigation (AA) and SC 3.2.4 Consistent Identification (AA): repeated navigation stays in the same order, and components with the same function are identified the same way. SC 3.2.6 Consistent Help (A): help mechanisms stay in the same relative place. The HIG is, in large part, a catalogue of the standards this heuristic tells you to follow.

In fomodoro. The primary button stays in exactly one place and changes only its label (Start, Pause, Resume), so the thumb always knows where to go. The gear is in the same corner on every screen. Use one verb per concept: if the timer "skips", the button says Skip, not Next or Advance. Steppers should look and behave like iOS steppers (minus on the left, plus on the right). On Mac, Command-comma should open Settings, and Space toggling start/pause matches media players. Warm tint always means focus and cool tint always means break; never reuse those tints for something else, such as errors.

H5. Error prevention HEURISTIC

Recap. Better than good error messages is a design that prevents the problem in the first place.

What it really means. Two kinds of error need different defences, a distinction from Don Norman [57]: *slips*, where you meant to do the right thing but your hand or attention went wrong (tapping Skip instead of Pause), and *mistakes*, where your mental model was wrong (thinking a settings change applies now). Slips are prevented with constraints, spacing, and good defaults; mistakes with clearer models and previews. For high-cost actions, prevention also means confirming or allowing undo.

Common misreadings. (1) "Prevention means confirmations everywhere." Over-confirming trains people to click OK without reading, which removes the protection exactly when it matters. (2) "Validate harder." Strict validation that rejects reasonable input is a different error (see Postel's law).

Real products.

- *Good:* email clients that notice you wrote "attached" but attached nothing, and ask before sending.
- *Good:* date pickers that grey out impossible dates, such as a return flight before the outbound one. *Bad:* a free-text date field that accepts anything and fails at submit.

Review question. What is the most likely slip on this screen, and what is the most likely misunderstanding? What does each cost, and does the layout make either impossible or cheap?

WCAG and HIG. Aligns with SC 3.3.4 Error Prevention (Legal, Financial, Data) (AA): for submissions with legal or financial effects, or that change user data, the action must be reversible, checked, or confirmed. SC 2.5.8 Target Size (Minimum) (AA, 24 by 24 CSS px) and Apple's 44 by 44 pt default both reduce slips. SC 3.3.7 Redundant Entry (A) prevents retyping errors.

In fomodoro. Steppers with minimum and maximum values make "2500 minutes" impossible. Skip and Reset are smaller and placed away from the primary button, so a hurried tap on Pause cannot hit Skip (Fitts's law, Section 5). The 6-digit code field uses `autocomplete="one-time-code"` and accepts paste, which prevents transcription slips. Two-factor setup should require entering a valid code from the authenticator app before 2FA is switched on, and offer recovery codes, which prevents the worst error in the app: locking yourself out. "Applies from your next block" prevents the mistake of believing the current block changed length.

H6. Recognition rather than recall HEURISTIC

Recap. Make objects, actions, and options visible so people do not have to remember information from one part of the interface to another.

What it really means. Recognizing something you see is far easier than retrieving it from memory unprompted; that is why multiple-choice exams feel easier than essay questions. The heuristic asks the interface to carry the memory load: show current values, recent items, what comes next, and what each control does. It is the design-side answer to the working-memory limit (Section 5).

Common misreadings. (1) "Put everything on screen." Recognition needs things to be findable, not all visible at once; a well-labelled menu supports recognition too. (2) "Icons support recognition." Only icons people already know (a gear, a share symbol). An unfamiliar icon forces recall of what it meant last time; label it.

Real products.

- *Good:* command palettes (Command-K in many apps, Command-Shift-P in VS Code) that let you type part of a command and recognize it in a list, with its keyboard shortcut shown beside it so you learn it in passing.
- *Bad:* a two-step form that asks you on step 2 to "confirm the amount entered earlier" without showing it, or a sign-in that makes you remember which of three methods you used last time.

Review question. What does the user need to know to use this screen correctly, and is all of it on the screen or one obvious step away?

WCAG and HIG. Aligns with SC 3.3.2 Labels or Instructions (A), SC 3.3.7 Redundant Entry (A: information already entered is auto-populated or available to select), and SC 3.3.8 Accessible Authentication (Minimum) (AA): sign-in must not depend on a cognitive function test such as remembering or transcribing, unless there is an alternative or a mechanism (such as paste and password managers) to help. 3.3.8 is H6 written into a standard.

In fomodoro. "Next: short break, 5 min" and the cycle dots mean you never have to count blocks in your head. Steppers display the current value. Sign-in should remember the email used last time on this device and offer the method used last time first. The code field allowing paste and iOS code autofill is not polish; it is what satisfies 3.3.8. On Mac, show keyboard shortcuts in tooltips or the menu so they can be recognized rather than memorized.

H7. Flexibility and efficiency of use HEURISTIC

Recap. Shortcuts, hidden from novices, can speed up expert interaction, so the design serves both.

What it really means. Novices and experts need different things from the same screen, and a good design lets people move from one to the other without a mode switch. Nielsen calls the shortcuts *accelerators*: keyboard shortcuts, gestures, defaults that remember your last choice, and personalization. The key word is *hidden*: accelerators must not clutter the novice path.

Common misreadings. (1) "Add a settings toggle for everything." Customization is one kind of flexibility, and the most expensive; smart defaults usually beat options. (2) "Gestures are shortcuts, so they can replace buttons." A gesture with no visible equivalent is not a shortcut, it is the only path, and it is invisible. (3) "Power users will find it." Accelerators need a discovery route: shortcut hints in menus or tooltips.

Real products.

- *Good:* Gmail's keyboard shortcuts, available for those who switch them on and invisible to everyone else.
- *Bad:* apps where a very frequent action is three taps deep every time, with no way to put it closer.

Review question. What will a daily user do on this screen dozens of times, and is there a faster path for it that a first-time user never has to see?

WCAG and HIG: a real tension. SC 2.1.1 Keyboard (A) aligns: everything must work from a keyboard. But SC 2.1.4 Character Key Shortcuts (A) constrains accelerators: if a shortcut uses only a letter, number, punctuation, or symbol key, users must be able to turn it off or remap it, or it must be active only while its component has focus. Single-key shortcuts collide with speech-input users, whose dictated

words arrive as keystrokes. The W3C Understanding document does not clearly classify the space bar [11], so Space-to-pause is a grey zone; single-letter shortcuts like R for Reset or S for Skip are clearly covered. WCAG wins: we recommend Command-modified shortcuts, or a setting to disable single-key ones. The HIG asks Mac apps to support standard keyboard shortcuts and to show them in menus.

In fomodoro. Space for start/pause, the installable PWA, and the Mac mini timer window are accelerators. Two concrete cautions. First, when keyboard focus is on the Reset button, pressing Space natively activates Reset; a global Space handler must not fire as well, or one keystroke does two things. Second, any letter shortcut needs an off switch. Signed-in settings are the flexibility layer; signed-out defaults (25/5/15, long break every 4) are the novice path, which is exactly the shape H7 wants.

H8. Aesthetic and minimalist design HEURISTIC

Recap. Interfaces should not contain information that is irrelevant or rarely needed; every extra unit of information competes with the relevant ones.

What it really means. It is about *signal-to-noise*, not about visual style. Nielsen's point is competition for attention: each element dilutes the visibility of the others. The test is relevance to the current task, not how few pixels are used. A dense professional tool can be minimalist if everything on screen is needed.

Common misreadings. (1) "Minimalist means flat, grey, and spacious." Low-contrast grey text and borderless controls are an accessibility failure, not minimalism. (2) "Remove labels; icons are cleaner." That trades a little visual noise for a lot of recall (H6).

Real products.

- *Good:* a search engine home page that is a logo and a box, because searching is the one task.
- *Bad:* early web portals that put news, weather, stocks, and ads on one page, so nothing could be found; or car touchscreens where climate controls compete with media and navigation for the same screen.

Review question. For each element, what happens if it is removed? If nothing bad happens for the main task, why is it here; if something bad happens, is it prominent enough?

WCAG and HIG: the most common conflict. Minimalism pushes toward pale text, faint borders, and icon-only buttons. WCAG pushes back with SC 1.4.3 Contrast (AA, 4.5:1 for normal text), SC 1.4.11 Non-text Contrast (AA, 3:1 for control boundaries and meaningful graphics), SC 1.4.1 Use of Color (A), SC 4.1.2 Name, Role, Value (A: icon buttons need an accessible name), and SC 2.5.3 Label in Name (A). WCAG wins every time. The HIG's emphasis on content over chrome (and, since 2025, on controls that float above content) aligns with H8, while its accessibility guidance carries the same contrast and labelling requirements.

In fomodoro. One primary button; settings behind the gear. But "Focus" stays as text even though the warm tint already signals focus, because color alone fails 1.4.1. The digits must meet 4.5:1 on both the warm and the cool tint (large text needs only 3:1, but the block name and "Next" line are normal-size). The ring is a meaningful graphic, so we recommend at least 3:1 between its track and its fill. Reset and Skip, if shown as icons, need accessible names, and we recommend visible text labels too, because neither has a universal icon.

H9. Help users recognize, diagnose, and recover from errors HEURISTIC

Recap. Error messages should use plain language, say precisely what went wrong, and suggest a fix.

What it really means. Three verbs, three jobs. *Recognize*: the error is noticeable, near where the problem is, and not conveyed by color alone. *Diagnose*: it says what happened in the user's terms. *Recover*: it offers the way forward, ideally as a button, and keeps everything the user already did.

Common misreadings. (1) "Be friendly." A cheerful "Oops!" with no diagnosis fails all three verbs. (2) "Show the technical detail for transparency." Error codes can be included for support, but after the plain explanation, not instead of it.

Real products.

- *Good:* payment forms that flag "card number is incomplete" inline beside the field the moment you leave it, with your other entries intact.
- *Bad:* forms that clear every field after one error, or a full-screen "Something went wrong" with no next step.

Review question. Trigger each error on purpose. Does the message say what happened, in plain words, where the user is looking, and offer the fix in one tap without losing their work?

WCAG and HIG. Close alignment: SC 3.3.1 Error Identification (A: the error is identified and described in text), SC 3.3.3 Error Suggestion (AA: suggest a correction when one is known), and SC 4.1.3 Status Messages (AA: errors announced to screen readers). The HIG's alert guidance asks for alerts that explain and offer clear actions.

In fomodoro. "Code expired. Send a new one." with the button beside it is the model. Others to design: a wrong code ("That code doesn't match. Check the newest email, or send a new code."); lost connection during handoff ("Can't reach your other devices right now. The timer keeps running here."), which also protects the user's real goal, the timer; notifications blocked in system settings ("Notifications are off for fomodoro. Here's how to turn them on.", with the steps for that platform). None of these should rely on red alone (1.4.1).

H10. Help and documentation HEURISTIC

Recap. Ideally the system needs no explanation; where help is needed, it should be easy to find, focused on the task, and short.

What it really means. Help is a fallback, and the best help is contextual: present at the moment and place of need, phrased as steps, about the user's task rather than the feature list. It was the one heuristic not among the nine in the 1994 paper's abstract (Section 2), which fits its status: needing it is a sign that the other nine have run out.

Common misreadings. (1) "Add an onboarding tour." Multi-screen tours shown before people have a task are mostly skipped and forgotten. (2) "Docs fix confusing UI." If a control needs a paragraph, redesign the control first.

Real products.

- *Good:* empty states that teach: a new project tool that, when the list is empty, explains in one line what goes here and offers the button to add the first item.
- *Bad:* help that lives only in a long FAQ page, or a tour of six tooltips that blocks the screen on first launch.

Review question. Where on this screen might someone get stuck, and is there a one-line, just-in-time answer at that exact spot?

WCAG and HIG. SC 3.2.6 Consistent Help (A): if help mechanisms (contact details, a help link, a chat) appear on multiple pages, they stay in the same relative order. SC 3.3.2 Labels or Instructions (A) covers the small inline instructions that are most of good help. The HIG asks that help be brief and in context.

In fomodoro. The install button opens an "Add to Home Screen" sheet only on iPhone Safari, with the two or three steps and a picture of the Share icon: help exactly when needed. Two-factor setup gets one line on what an authenticator app is. "Take over here?" may deserve a one-line explanation the first time it appears. If we ever add a help link, it lives in the same place in Settings on every platform (3.2.6).

4. Running a heuristic evaluation solo

The method, and what the evidence says MODERATE

Heuristic evaluation is an *inspection* method: evaluators examine an interface against the heuristics and list violations, without users. It is cheap and fast, which is why it spread. Its weakness was visible in the very first paper: across four experiments, individual evaluators found only 20% to 51% of the known problems, and pooling several evaluators found far more [1]. Nielsen's 1992 study of six cases put a single evaluator's average detection at about 42% of major problems and 32% of minor ones, with usability specialists outperforming novices and specialists who also knew the product domain doing best [6]. NN/g today recommends three to five independent evaluators, each working alone for one to two hours in two passes [9].

The evaluator effect STRONG

Hertzum and Jacobsen reviewed 11 studies of three methods (heuristic evaluation, cognitive walkthrough, and think-aloud testing) and found that different evaluators using the same method on the same system report markedly different problems [8]. Average agreement between any two evaluators ranged from 5% to 65%. The effect held for novices and experts, for cosmetic and severe problems, for simple and complex systems, and for severity judgments as well as detection. No method was consistently better. Their title called it a chilling fact, and the practical meaning is plain: **one person's review is a sample, not an audit**. Each evaluator notices problems shaped by their own habits and blind spots, so adding evaluators adds problems found, with diminishing returns.

Doing it alone anyway

For fomodoro there is one evaluator, you. You cannot remove the evaluator effect, but you can reduce it by making yourself several slightly different evaluators. We recommend:

1. **Scope and tasks.** Pick one flow, not the app. Write the tasks first: "Start a focus block on iPhone," "Change the long break to 20 minutes," "Sign in on the Mac and take over the running timer," "Install on iPhone."
2. **Pass 1, familiarize.** Do the tasks once as a user, noting nothing.
3. **Pass 2, inspect.** Go screen by screen with the review questions from Section 3. For each problem, record: where, what you observed, which heuristic(s) or WCAG criterion, and a fix idea. One problem per row.
4. **Vary the evaluator.** Repeat pass 2 on a different day and device: once on iPhone one-handed, once on Mac keyboard-only, once with VoiceOver on, once at 200% text size. Each is effectively a different evaluator with different blind spots.
5. **Rate severity later**, not during inspection (next section).
6. **Add a second evaluator if you can**: a friend for 20 minutes, or an AI reviewer with the same tasks. Either is another sample, not user evidence.
7. **Run a WCAG check separately.** Heuristic evaluation is not an accessibility audit; automated checkers plus a manual keyboard and screen-reader pass cover what the heuristics only gesture at.

Severity ratings HEURISTIC

Nielsen's 1994 scale is the standard way to prioritize the list [7]. Severity combines *frequency* (how often it happens), *impact* (how hard it is to overcome), and *persistence* (one-time, or every time), with market impact as a further factor. Nielsen advises rating after the inspection, not during, and notes that one evaluator's ratings are unreliable; the mean of about three is satisfactory for practical purposes [7]. Solo, rating on a different day from inspecting is the closest substitute.

Rating	Meaning	fomodoro example (hypothetical)
0 Not a problem	Not a usability problem at all, on reflection.	The ring drains anticlockwise; you expected clockwise but it reads fine.
1 Cosmetic	Fix only if there is spare time.	Cycle dots are 1 px off center on Mac.
2 Minor	Low priority.	No "Applies from your next block" note; users are briefly confused, then figure it out.
3 Major	High priority; important to fix.	Skip sits next to Pause; a hurried tap ends the block early several times a week.
4 Catastrophe	Must fix before release.	The code field blocks paste, so some users cannot sign in (also fails WCAG 3.3.8).

Rule of thumb for this project: any WCAG AA failure is at least a 3, whatever the heuristic severity suggests, because WCAG sits above heuristics in the authority order.

5. The laws of UX in depth

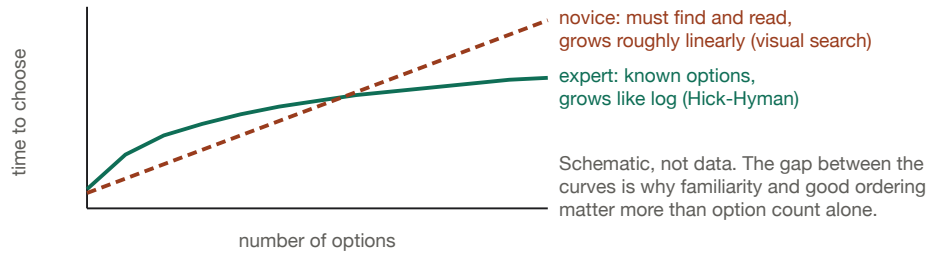
The primer warned that the "laws", as collected by Yablonski [13], are a mixed bag, and that the label of each one matters more than its name. Here each gets what was actually measured, where it holds, where it breaks, how it is misused, and its job in fomodoro. A useful habit: whenever someone cites a law in a review, ask *what was measured, on whom, doing what?* The answer usually tells you how far the law stretches.

Hick-Hyman law STRONG IN THE LAB MODERATE AS A UI RULE

The original finding. Hick (1952) had people respond to one of up to ten lamps by pressing the matching key; reaction time rose with the logarithm of the number of alternatives, roughly `RT = a + b · log2(n + 1)`, where the +1 allows for the possibility of no signal [14]. Hyman (1953) showed the real driver is *information* in the information-theory sense: when some alternatives are more likely than others, reaction time tracks the average uncertainty in bits, not the raw count [15]. So what was measured: practiced people, a fixed and known set of stimulus-response pairs, and the time to *decide*, with no searching or reading.

Where it holds and where it breaks. It holds when people already know the options and their positions: an expert choosing a familiar menu item. It breaks when people must first find and read the options [16]. Cockburn, Gutwin, and Greenberg's menu model treats these as two regimes: novices search visually, item by item, so time grows roughly linearly with the number of items; experts who know where things are decide in Hick-Hyman time [18]. Interestingly, the classic menu study usually cited against long menus points the other way: Landauer and Nachbar found that for menus ordered alphabetically or numerically, time per screen grew only logarithmically with the number of items, so broad shallow menus beat narrow deep ones [17].

> **Correction to the primer.** The primer cited Landauer and Nachbar (its [9]) for the claim that scanning an unfamiliar menu grows closer to linearly. Their result was logarithmic, for ordered menus. The linear-search point is better supported by Cockburn and colleagues' model [18]. The primer's conclusion (treat Hick as a direction, not a formula) stands.



How it is misused. As "fewer options is always better", which leads to hiding things in deep hierarchies, where people pay several decision-plus-search costs instead of one. Also as an argument against features: Hick is about choices *at one moment*, not about how much an app can do.

In fomodoro. The timer screen offers one main decision (go or pause), with two secondary ones visually demoted. Settings is one flat screen of four steppers plus notification and account sections, not nested

pages: broad and shallow, ordered as you experience a cycle.

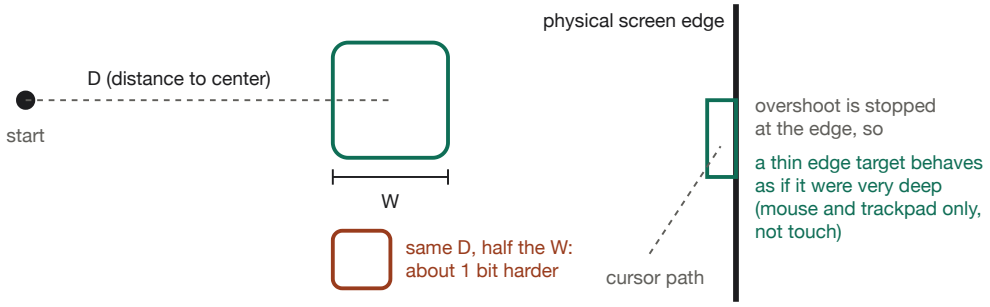
Fitts's law STRONG

The original finding. Fitts (1954) had people tap alternately between two metal strips with a stylus, varying the strips' width and the distance between them [19]. Movement time rose linearly with an *index of difficulty* that combines the two. The intuition: aiming is a feedback process. You make a fast initial movement and then correct, and the number of corrections depends on how big your target is relative to how far you travelled. Doubling the distance costs about as much as halving the width.

The formula, in the version HCI uses. MacKenzie proposed the *Shannon formulation*, by analogy with Shannon's channel-capacity theorem [20, 21]:

```
MT = a + b · log2(D / W + 1)
```

MT is movement time, D the distance to the target's center, W the target's width along the direction of movement, and a and b constants fitted for a given device and person. The log term is the index of difficulty, in bits. Fitts's original version used `log2(2D / W)`; the Shannon form fits data slightly better and never goes negative for very close, large targets. A later standard practice uses the *effective* width, derived from where people's taps actually land (about 4.13 times the standard deviation of the endpoints), so that the model accounts for people trading speed for accuracy [22].



A worked example. Thumb resting 200 pt from a primary button 88 pt wide: `log2(200/88 + 1) ≈ 1.7` bits. A 44 pt Skip button at the same distance: about 2.5 bits. That difference is small in milliseconds, but error rates climb as difficulty rises when people move fast, and on the timer screen, errors are what matter.

Edges and corners. On a desktop, the pointer stops at the physical screen edge, so a target touching the edge can absorb any overshoot and behaves as if it were very deep in that direction. The macOS menu bar and hot corners exploit this; work on menu selection by Walker, Smelcer, and Nilsen is commonly credited with describing this *impermeability* of the edge [23]. Two limits: it applies only at the *physical* screen boundary (a browser window's edge does not stop the pointer unless the window is full screen), and it does not apply to touch at all.

Touch versus cursor. Fingers differ from pointers in three ways. First, a finger has an absolute imprecision that does not shrink as you slow down; Bi, Li, and Zhai's FFitts model adds a term for it, which is why very small touch targets do worse than classic Fitts predicts [24]. Second, touch points are systematically offset from where people think they touched, depending on finger angle [25]. Third, there is no hover and no edge-stopping. On iPhone, screen edges are where system gestures live (Home,

Control Center, Back), and the HIG asks apps not to compete with them [12]. This is a direct conflict between the desktop edge heuristic and the HIG, and the HIG wins on iPhone. Thumb-target studies point to roughly 7 to 10 mm as a comfortable size [26], consistent with Apple's 44 pt default; WCAG 2.5.8's 24 CSS px is the legal floor.

How it is misused. As "make everything big", which defeats itself (if everything is big, distances grow and nothing is prominent). Or ignoring that Fitts's law models time to hit an *intended* target; it says nothing about accidentally hitting a *neighbour*, which is about spacing.

In fomodoro. The primary button is the largest target and sits in the lower-middle of the iPhone screen, where thumbs rest, clear of the home indicator's gesture zone (safe areas). Reset and Skip are smaller and separated from it by more than a finger's width, because missing Pause is cheap and hitting Skip by accident is not. On the Mac mini window, all targets keep at least 24 px with spacing, and we do not rely on edge effects because the window is not full screen.

Working memory: Miller to Cowan STRONG

The original finding. Miller's 1956 paper reviewed two separate lines of research: *absolute judgment* (how many levels of one dimension, such as pitch, people can reliably identify: about seven) and *immediate memory span* (how many items people can repeat back: also about seven) [27]. His more important point was that span is limited in *chunks*, not bits: recoding items into larger meaningful units lets people hold far more information. He treated the recurrence of seven as close to a coincidence. Cowan (2001) reviewed studies that prevented rehearsal and chunking and concluded that the true capacity is about 3 to 5 chunks, typically around four [28].

Where it holds and where it breaks. It holds for holding *new, unrelated* information in mind for seconds: a code, a number, a path you were just told. It does not apply to things that stay on screen. A visible menu of twelve items does not tax working memory; it taxes visual search (Hick-Hyman, novice regime).

How it is misused. The famous one: "menus and navigation bars should have at most 7 ± 2 items". That misreads both the number (it is nearer four) and the scope (visible items are recognized, not remembered). The useful version is H6: do not make people carry information across screens.

In fomodoro. The 6-digit sign-in code is exactly the kind of thing working memory handles badly when copied by hand between apps. Display it in the email as two groups of three (123 456), which makes two chunks, and support autofill and paste so nobody has to hold it at all. Cycle dots and "Next" mean no one counts blocks in their head.

Jakob's law HEURISTIC

The original claim. Nielsen observed in 2000 that users spend most of their time on other sites, so they come to yours expecting it to work like those [29]. It is an argument from experience and economics, not an experiment.

Where it holds and where it breaks. It holds strongly for common patterns: sign-in, settings, search, shopping carts, standard gestures. It breaks where the convention is bad (copying a poor norm transfers the problem), and where a product genuinely does something new, in which case the job is to borrow the nearest familiar model.

How it is misused. As "copy the market leader", including its deceptive patterns; or as a veto on any improvement. It is a strong default, not a ban on better ideas.

In fomodoro. Settings should look like iOS Settings: grouped rows, steppers, switches. Email-code sign-in and "Sign in with Google" are patterns people know. Handoff borrows from music apps' "play on this device" model: one device is playing, others show what is playing and offer to take over.

Doherty threshold HEURISTIC

The original claim. Doherty and Thadhani's 1982 IBM report argued that productivity rises sharply when system response time falls below about a second, and the 400 ms figure popularly attached to it is attributed to this report [30]; as the primer noted, we could not confirm the exact figure in the primary text. The study context was mainframe programmers, and IBM sold the faster systems. Separately, the classic response-time limits (about 0.1 s feels instantaneous, about 1 s keeps the flow of thought, about 10 s holds attention) come from Robert Miller's 1968 analysis as popularized by Nielsen [31, 32].

Where it holds and where it breaks. The direction (faster feedback feels better and keeps people in flow) is broadly supported. The specific threshold is not a law and was measured in a very different setting.

How it is misused. As a reason to add animation that *looks* fast while the work is slow, or to fake completion. Optimistic UI is fine when the action almost always succeeds and failure is reported honestly.

In fomodoro. Pressing Start must change the button and start the ring within about 100 ms, locally, before any network call. Handoff sync can take a second or more across devices; the local device shows its state immediately and other devices catch up. The primer's Core Web Vitals target (INP at or below 200 ms) is the measurable version of this.

Tesler's law (conservation of complexity) HEURISTIC

The original claim. Larry Tesler, in an interview published by Dan Saffer, argued that every application has an inherent amount of complexity that cannot be removed, only moved: either the system (and its developers) handles it or the user does [33]. Better for one developer to take on extra work than for millions of users to each carry it.

Where it holds and where it breaks. It holds as a design ethic and a useful question ("who is carrying this?"). It breaks if taken to mean all complexity is irreducible: some complexity is accidental and can simply be deleted, along with the feature that caused it.

How it is misused. As an excuse ("it's inherently complex") for a confusing screen, or the opposite: hiding complexity the user actually needs to manage, such as cost or consequences.

In fomodoro. The system carries clock differences between devices: every device computes the remaining time from one shared end time, so the user just sees the same countdown everywhere. Some complexity cannot be fully absorbed: on iPhone, web push works only for web apps added to the Home Screen, so the user must install. The install sheet is how we carry as much of that as we can.

Goal-gradient effect MODERATE

The original finding. Hull (1932) observed that rats in a maze ran faster as they neared the food [34]. Kivetz, Urminsky, and Zheng (2006) found the human version in the field: coffee-shop customers with a stamp card bought coffee more frequently as they approached the free drink, and a 12-stamp card with two stamps already filled was completed faster than a blank 10-stamp card, although both needed ten purchases [35]. That second result, illusory progress, is the one designers copy.

Where it holds and where it breaks. Bonezzi, Brendl, and De Angelis found a "stuck in the middle" pattern: motivation can be high near the start and near the end, and lowest in the middle, depending on whether people focus on progress made or distance remaining [36]. So "effort rises toward the goal" is a common pattern, not a universal law.

How it is misused. Pre-filled progress bars and pretend head starts used to manipulate, and streak counters that turn motivation into guilt. Both drift into deceptive design.

In fomodoro. The ring and the cycle dots make the finish line visible honestly. "Stuck in the middle" suggests the third focus block of four is the hard one, and the dots show you are past halfway. We recommend against streaks or guilt messages: the app should help you focus, not shame you for a day off.

Zeigarnik effect WEAK and Ovsiankina effect MODERATE

The original finding. Zeigarnik (1927) reported that people interrupted during simple tasks later recalled those tasks better than ones they had completed [37]. Her colleague Ovsiankina (1928) studied something different: interrupted people tended to go back and *resume* the task when given the chance [38].

What the evidence says now. Ghibellini and Meier's 2025 meta-analysis found no reliable memory advantage for unfinished tasks, but did support the tendency to resume [39]. So the popular claim (unfinished things nag at memory) is the weak one; the less famous one (people want to finish what they started) holds up better.

How it is misused. Profile-completeness meters and half-filled checklists designed to "nag" people back. The memory mechanism behind that idea is not supported.

In fomodoro. Design for Ovsiankina, not Zeigarnik: make resuming effortless. If the app is closed mid-block, reopening it shows the block still running (or paused) exactly where it was. Resume is the same button in the same place. No "you left a block unfinished" notifications.

Peak-end rule STRONG

The original finding. In Kahneman and colleagues' 1993 study, people held a hand in painfully cold water for 60 seconds in one trial, and in another for the same 60 seconds followed by 30 more seconds at a slightly less cold temperature. Asked which to repeat, most chose the *longer* trial, because it ended better [40]. Fredrickson and Kahneman showed the same *duration neglect* with film clips: retrospective ratings tracked the most intense moment and the end, with duration barely mattering [41].

Where it holds and where it breaks. Alaybek and colleagues' 2022 meta-analysis (174 effect sizes) found strong support for a peak-end effect on retrospective evaluations and very little effect of duration

[42]. It also found that the simple average of the whole experience predicted evaluations about as well as peak and end did, and a 2024 corrigendum revised some estimates. So: peaks and ends matter a lot, but the middle is not irrelevant. And what was measured is retrospective judgment of a single episode; applying it to an app used daily is our inference.

How it is misused. As permission for a bad middle ("the ending will save it"), or as a reason for manipulative "delight" moments.

In fomodoro. The end of each block, and especially the long break at the end of a cycle, is the moment to get right: a pleasant chime, a calm color shift from warm to cool, a clear "Break time". The peak is the risk: a jarring alarm or a failed handoff will be remembered more than twenty smooth minutes. And the average matters too, so the whole block should be calm.

Aesthetic-usability effect MODERATE

The original finding. Kurosu and Kashimura asked people to rate many layout variants of an ATM screen for beauty and for how easy they looked to use; apparent usability correlated more strongly with apparent beauty than with the designs' inherent usability [43]. Tractinsky and colleagues found that after people actually used a system, perceived beauty still shaped perceived usability, while the system's real usability did not change their ratings much [44]. Hassenzahl found beauty related mostly to hedonic qualities (identity, stimulation) and only weakly to usability [45]. Sonderegger and Sauer found that a more attractive phone prototype received higher perceived usability ratings and, in that study, also better task performance [46].

Where it holds and where it breaks. It holds for *perceived* usability and first impressions. It does not show that beauty makes an interface easier to use in general, and the strength of the link varies by context.

How it is misused. Polishing first and testing never; reading satisfaction scores as evidence of usability.

In fomodoro. The calm warm and cool palette is worth the effort. But when testing your own app, count errors and hesitations (did I tap Skip by mistake? did I wonder whether settings saved?) rather than trusting the feeling that it is nice to use: you are the evaluator most biased by its looks.

Von Restorff (isolation) effect STRONG

The original finding. von Restorff (1933) showed that in a list of similar items, an item that differs (a number among syllables, say) is remembered better [47]. Hunt's re-reading of the original work argues the effect is about distinctiveness relative to the surrounding context in memory, not simply about the odd item grabbing attention at the moment it appears [48].

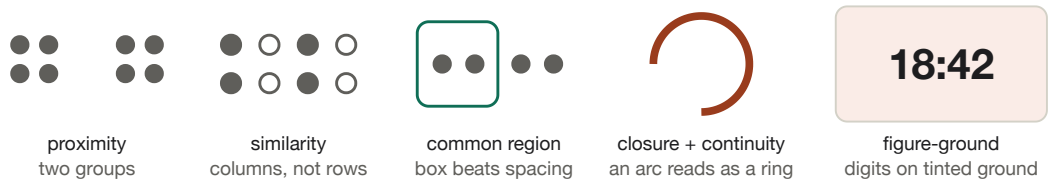
Where it holds and where it breaks. It is a memory finding about lists. Using it to make a button stand out is an extension (a reasonable one, supported by everyday experience of visual pop-out, but not what was tested). It breaks as soon as several things are made distinctive: then none is.

How it is misused. Making everything "pop", and distinguishing by color alone, which fails WCAG 1.4.1 and disappears for color-blind users.

In fomodoro. Exactly one filled, high-contrast control: the primary button. Reset, Skip, and the gear are plain. The difference is in size, fill, and position, not only hue.

Gestalt principles STRONG

The original findings. Wertheimer (1923) described how people spontaneously group visual elements: by **proximity** (near things group), **similarity** (alike things group), **continuity** or good continuation (elements on a smooth line or curve are seen as one path), and **closure** (we complete gaps to see whole shapes) [49]. Rubin (1915) described **figure-ground**: we see one region as an object in front and the rest as background [50]. Palmer (1992) added **common region**: elements inside a shared boundary group together, often more strongly than by proximity [51]. A century of research reviewed by Wagemans and colleagues confirms these as real perceptual tendencies, while showing they interact and compete rather than working as independent rules [52].



Where they hold and where they break. They hold as descriptions of perception. They break as a design tool when cues conflict: a box around the wrong items (common region) overrides careful spacing (proximity), so the user sees the box's grouping.

WCAG alignment. A visual group is invisible to a screen reader unless it is in the markup. SC 1.3.1 Info and Relationships (A) requires that structure conveyed visually is also available programmatically: headings, lists, `<fieldset>` with `<legend>`, grouped controls. SC 1.3.2 Meaningful Sequence (A) and 2.4.3 Focus Order (A) ask that reading and focus order follow the visual grouping. Gestalt tells you what sighted users perceive; WCAG makes you encode it for everyone.

How they are misused. Adding borders and cards around everything (common region overload), or grouping by color alone.

In fomodoro. Digits, block name, and the "Next" line sit close together inside the ring (proximity and common region): one unit, "where am I". Reset and Skip sit as a pair, apart from the primary button. Cycle dots use similarity, filled for done and outlined for remaining, which differs in shape as well as color. The draining ring relies on closure: a shrinking arc still reads as part of a circle. The tinted background is ground; the digits are figure, which is why they need strong contrast. In Settings, the four duration steppers form one group with a visible heading, and in the markup one `<fieldset>` with a `<legend>`.

Serial position effect STRONG

The original finding. When people recall a list of words in any order, they remember the first few (primacy) and the last few (recency) best and the middle worst; Murdock's 1962 study mapped this U-shaped curve across list lengths [53]. Glanzer and Cunitz showed the two ends have different causes: a short distracting delay before recall wipes out the recency advantage but leaves primacy intact [54].

Where it holds and where it breaks. It holds for recalling sequences presented one after another. Its application to navigation menus ("put the important items first and last") is a plausible extension with

little direct testing, and it is weaker when items stay visible, because people can just look.

How it is misused. As a strong rule for ordering visible menus, or to justify burying unwelcome terms in the middle of a long text, which is a deceptive pattern.

In fomodoro. It matters most for things people hear or read once: VoiceOver announcements and short instructions. Put the key information first ("Short break, 5 minutes") rather than after a preamble. In the install sheet, the critical step (tap Share) comes first; the confirmation step comes last. In Settings, the most-changed value (focus length) goes first.

Postel's law (robustness principle) HEURISTIC

The original claim. Not from UX at all: from the 1980 TCP specification, which advised implementers to "be conservative in what you do, be liberal in what you accept from others" [55]. Designers adopted it as: accept messy human input, produce clean, consistent output.

Where it holds and where it breaks. In UX it holds well for formatting tolerance (spaces, dashes, capitalization) where the intent is unambiguous. It breaks where tolerance means guessing. The networking community itself has reconsidered: RFC 9413 (2023) argues that tolerating faulty input lets errors become entrenched and can cause security problems, and that protocols should be actively maintained instead [56]. The UX version: silently interpreting ambiguous input can do the wrong thing confidently.

How it is misused. Accepting anything without feedback, so the user never learns that "1.5" became 15 minutes.

In fomodoro. The code field accepts "123 456", "123-456", or a pasted line containing the code, and keeps only the six digits. Email addresses are trimmed and compared case-insensitively. Durations do not need Postel because steppers remove free input altogether (H5): the most robust input is the one that cannot be malformed. Where a guess would be needed, ask instead (WCAG 3.3.3, Error Suggestion).

6. Review checklist for a fomodoro screen

One screen at a time. A and AA items are WCAG 2.2 requirements and outrank everything else (a failure is severity 3+). Log each problem with location, heuristic or law, and severity 0 to 4.

Check	Ask	Source
Status at a glance	Frozen at a random moment, does the screen say what state it is in, what just happened, and what comes next? Is a read-only handoff view labelled? Is it clear that settings apply from the next block, and that signed-out mode uses defaults?	H1, H5
Status for everyone	Are block changes and errors announced to VoiceOver, without per-second chatter or focus jumps?	AA 4.1.3, H1
Words	Any label a non-developer would ask about (TOTP, session, interval)?	H2, AA 2.4.6
Exits and undo	Can every state be left? Are Reset, Skip, and take-over undoable or confirmed in proportion to cost? Do taps fire on release?	H3, A 2.5.2
Consistency	Primary button in the same place with the same verbs? Gear in the same corner? Tints used only for focus and break? iOS and Mac conventions followed?	H4, Jakob, AA 3.2.3/3.2.4
Likely slip	What is the most likely mis-tap, and does size and spacing make it hard? Primary target large, destructive ones small and apart, all at least 44 pt on iPhone?	H5, Fitts, AA 2.5.8, HIG
Memory load	Is everything the user needs visible or one step away? Can codes be pasted or autofilled?	H6, working memory, AA 3.3.8
Accelerators	Is there a faster path for frequent actions? Can single-key shortcuts be turned off? Does Space avoid double-firing on a focused button?	H7, A 2.1.4, A 2.1.1
Noise	Can any element go without harming the main task? Is exactly one thing visually dominant?	H8, Hick, Von Restorff
Contrast and color	Text at least 4.5:1 on both tints, ring and control edges at least 3:1, nothing conveyed by color alone, icon buttons named?	AA 1.4.3, AA 1.4.11, A 1.4.1, A 4.1.2
Errors	Trigger each one. Plain words, next to the problem, with a one-tap fix, input preserved?	H9, A 3.3.1, AA 3.3.3
Help in context	Where might someone get stuck, and is there one line of help right there? Help in the same place everywhere?	H10, A 3.2.6
Grouping	Do spacing and boundaries group the right things, and is each visual group also a group in the markup?	Gestalt, A 1.3.1
Speed	Does every tap get visible feedback within about 100 ms, locally, before any network call?	Doherty, INP
Endings	Is the end of a block and of a cycle pleasant, and is the worst moment (alarm, failed sync) handled gently?	Peak-end
Honest progress	Is progress shown truthfully, with no fake head starts, streak guilt, or nagging about unfinished blocks? Is resuming one tap?	Goal-gradient, Ovsiankina
Who carries complexity	Is anything hard here that the system could absorb (time sync, input formats, platform quirks)?	Tesler, Postel
Bias check	Did I test with my keyboard, VoiceOver, one hand, and large text, and count errors rather than trust how it feels?	Evaluator effect, aesthetic-usability

7. Where this goes next

Heuristics and laws are tools for *inspecting* a design. The evaluator effect in Section 4 is the reminder that inspection alone is not enough: at some point you need to watch real use. The next document in the series, **usability-fundamentals.pdf**, picks up there: what usability actually measures (effectiveness, efficiency, satisfaction), how to run small usability tests and dogfooding sessions, and how to turn what you observe into fixes. Bring this checklist with you; it becomes the list of things to watch for.

Sources and evidence strength

Evidence summary

Concept	Label	Why
Nielsen's ten heuristics (each of H1 to H10)	HEURISTIC	Synthesized from existing principles and a problem database; explanatory, not experimentally validated rules [1, 2, 3, 5].
Heuristic evaluation as a method	MODERATE	Studied empirically; single evaluators find a minority of problems [1, 6, 9].
Evaluator effect	STRONG	Consistent across 11 studies, three methods, novices and experts [8].
Severity rating scale	HEURISTIC	Practitioner scale; single-rater reliability known to be low [7, 8].
WCAG 2.2 criteria cited	STRONG	Formal W3C standard [10, 11].
Apple HIG	HEURISTIC	Authoritative platform guidance, not evidence [12].
Hick-Hyman law	STRONG / MODERATE	Robust for practiced choice; menus add search, which behaves differently [14, 15, 16, 17, 18].
Fitts's law	STRONG	Decades of replication; touch needs extensions [19, 20, 21, 22, 24].
Edge and corner targets	MODERATE	Follows from Fitts for pointers; direct study known to us only via secondary sources [23]. Not applicable to touch.
Touch target size (7 to 10 mm, 44 pt)	MODERATE	Thumb studies plus platform default [12, 25, 26].
Working-memory limit (about 4 chunks)	STRONG	Well established; "7" superseded [27, 28].
Jakob's law	HEURISTIC	Practitioner observation [29].
Doherty threshold	HEURISTIC	1982 vendor report; 400 ms figure unconfirmed in primary text [30, 31, 32].
Tesler's law	HEURISTIC	Practitioner principle [33].
Goal-gradient effect	MODERATE	Animal and field evidence; middle slump shows boundaries [34, 35, 36].

Zeigarnik effect	WEAK	2025 meta-analysis: no reliable recall advantage [37, 39].
Ovsiankina effect	MODERATE	Resumption tendency supported by the same meta-analysis [38, 39].
Peak-end rule	STRONG	Experiments plus meta-analysis; overall average also predictive [40, 41, 42].
Aesthetic-usability effect	MODERATE	Mostly perceived usability; context-dependent [43, 44, 45, 46].
Von Restorff effect	STRONG	Replicated memory finding; attention use is extension [47, 48].
Gestalt principles	STRONG	A century of perception research; principles interact [49, 50, 51, 52].
Serial position effect	STRONG	Replicated list-recall finding; UI menu use is extrapolation [53, 54].
Postel's law	HEURISTIC	Engineering maxim, now contested in its home field [55, 56].
Slips versus mistakes	HEURISTIC	Influential framework [57].

References

1. Nielsen, J., & Molich, R. (1990). Heuristic evaluation of user interfaces. *Proc. CHI '90*, 249–256. ACM. doi:10.1145/97243.97281
2. Molich, R., & Nielsen, J. (1990). Improving a human-computer dialogue. *Communications of the ACM*, 33(3), 338–348. doi:10.1145/77481.77486
3. Nielsen, J. (1994). Enhancing the explanatory power of usability heuristics. *Proc. CHI '94*, 152–158. ACM. doi:10.1145/191666.191729
4. Nielsen, J. (1994, updated since). 10 usability heuristics for user interface design. Nielsen Norman Group. nngroup.com/articles/ten-usability-heuristics
5. Nielsen, J. (2024, February 15). How I developed the 10 usability heuristics. *Jakob Nielsen on UX* (Substack). jakobnielsenphd.substack.com/p/usability-heuristics-history
6. Nielsen, J. (1992). Finding usability problems through heuristic evaluation. *Proc. CHI '92*, 373–380. ACM. doi:10.1145/142750.142834. Detection figures as summarized at nngroup.com/articles/usability-problems-found-by-heuristic-evaluation
7. Nielsen, J. (1994, November 1). Severity ratings for usability problems. Nielsen Norman Group. nngroup.com/articles/how-to-rate-the-severity-of-usability-problems
8. Hertzum, M., & Jacobsen, N. E. (2001). The evaluator effect: A chilling fact about usability evaluation methods. *International Journal of Human-Computer Interaction*, 13(4), 421–443. doi:10.1207/s15327590ijhc1304_05 (reprinted 2003, 15(1), 183–204)
9. Nielsen Norman Group (2023, June 25). How to conduct a heuristic evaluation. nngroup.com/articles/how-to-conduct-a-heuristic-evaluation
10. W3C (2023; updated 12 Dec 2024). *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C Recommendation. w3.org/TR/WCAG22
11. W3C. *Understanding WCAG 2.2*, including Understanding SC 2.1.4 Character Key Shortcuts. w3.org/WAI/WCAG22/Understanding (accessed Nov 2026)
12. Apple. *Human Interface Guidelines*: Accessibility; Buttons; Gestures; Layout; Motion; Writing. developer.apple.com/design/human-interface-guidelines (accessed Nov 2026)
13. Yablonski, J. (2020). *Laws of UX*. O'Reilly Media (2nd ed. 2024). lawsofux.com
14. Hick, W. E. (1952). On the rate of gain of information. *Quarterly Journal of Experimental Psychology*, 4(1), 11–26. doi:10.1080/17470215208416600
15. Hyman, R. (1953). Stimulus information as a determinant of reaction time. *Journal of Experimental Psychology*, 45(3), 188–196. doi:10.1037/h0056940

16. Seow, S. C. (2005). Information theoretic models of HCI: A comparison of the Hick-Hyman law and Fitts' law. *Human-Computer Interaction*, 20(3), 315–352.
17. Landauer, T. K., & Nachbar, D. W. (1985). Selection from alphabetic and numeric menu trees using a touch screen: Breadth, depth, and width. *Proc. CHI '85*, 73–78. doi:10.1145/317456.317470 (finding as reported in secondary sources; full text not read)
18. Cockburn, A., Gutwin, C., & Greenberg, S. (2007). A predictive model of menu performance. *Proc. CHI '07*, 627–636. doi:10.1145/1240624.1240723
19. Fitts, P. M. (1954). The information capacity of the human motor system in controlling the amplitude of movement. *Journal of Experimental Psychology*, 47(6), 381–391.
20. MacKenzie, I. S. (1989). A note on the information-theoretic basis for Fitts' law. *Journal of Motor Behavior*, 21(3), 323–330. doi:10.1080/00222895.1989.10735486
21. MacKenzie, I. S. (1992). Fitts' law as a research and design tool in human-computer interaction. *Human-Computer Interaction*, 7(1), 91–139.
22. Soukoreff, R. W., & MacKenzie, I. S. (2004). Towards a standard for pointing device evaluation: Perspectives on 27 years of Fitts' law research in HCI. *International Journal of Human-Computer Studies*, 61(6), 751–789. doi:10.1016/j.ijhcs.2004.09.001
23. Walker, N., Smelcer, J. B., & Nilsen, E. (1991). Optimizing speed and accuracy of menu selection: A comparison of walking and pull-down menus. *International Journal of Man-Machine Studies*, 35(6), 871–890. (Edge "impermeability" attribution via secondary sources; full text not read. See also Walker & Smelcer, *Proc. CHI '90*, 221–226, doi:10.1145/97243.97277.)
24. Bi, X., Li, Y., & Zhai, S. (2013). FFitts law: Modeling finger touch with Fitts' law. *Proc. CHI '13*, 1363–1372. doi:10.1145/2470654.2466180
25. Holz, C., & Baudisch, P. (2010). The generalized perceived input point model and how to double touch accuracy by extracting fingerprints. *Proc. CHI '10*, 581–590. doi:10.1145/1753326.1753413
26. Parhi, P., Karlson, A. K., & Bederson, B. B. (2006). Target size study for one-handed thumb use on small touchscreen devices. *Proc. MobileHCI '06*, 203–210.
27. Miller, G. A. (1956). The magical number seven, plus or minus two. *Psychological Review*, 63(2), 81–97.
28. Cowan, N. (2001). The magical number 4 in short-term memory: A reconsideration of mental storage capacity. *Behavioral and Brain Sciences*, 24(1), 87–114.
29. Nielsen, J. (2000, July 23). End of web design. Nielsen Norman Group. nngroup.com/articles/end-of-web-design
30. Doherty, W. J., & Thadhani, A. J. (1982). *The economic value of rapid response time*. IBM report GE20-0752-0. (Not read in full; 400 ms figure unconfirmed.)
31. Miller, R. B. (1968). Response time in man-computer conversational transactions. *Proc. AFIPS Fall Joint Computer Conference*, 33, 267–277.
32. Nielsen, J. (1993). *Usability Engineering*, ch. 5. Academic Press. Summary: nngroup.com/articles/response-times-3-important-limits
33. Saffer, D. (2006). *Designing for Interaction* (interview with Larry Tesler on the law of conservation of complexity). New Riders.
34. Hull, C. L. (1932). The goal-gradient hypothesis and maze learning. *Psychological Review*, 39(1), 25–43.
35. Kivetz, R., Urminsky, O., & Zheng, Y. (2006). The goal-gradient hypothesis resurrected. *Journal of Marketing Research*, 43(1), 39–58.
36. Bonezzi, A., Brendl, C. M., & De Angelis, M. (2011). Stuck in the middle: The psychophysics of goal pursuit. *Psychological Science*, 22(5), 607–612. doi:10.1177/0956797611404899
37. Zeigarnik, B. (1927). Das Behalten erledigter und unerledigter Handlungen. *Psychologische Forschung*, 9, 1–85.
38. Ovsiankina, M. (1928). Die Wiederaufnahme unterbrochener Handlungen. *Psychologische Forschung*, 11, 302–379.
39. Ghibellini, R., & Meier, B. (2025). Interruption, recall and resumption: A meta-analysis of the Zeigarnik and Ovsiankina effects. *Humanities and Social Sciences Communications*, 12, 962. doi:10.1057/s41599-025-05000-w
40. Kahneman, D., Fredrickson, B. L., Schreiber, C. A., & Redelmeier, D. A. (1993). When more pain is preferred to less: Adding a better end. *Psychological Science*, 4(6), 401–405.
41. Fredrickson, B. L., & Kahneman, D. (1993). Duration neglect in retrospective evaluations of affective episodes. *Journal of Personality and Social Psychology*, 65(1), 45–55.
42. Alaybek, B., Dalal, R. S., Fyffe, S., Aitken, J. A., et al. (2022). All's well that ends (and peaks) well? A meta-analysis of the peak-end rule and duration neglect. *Organizational Behavior and Human Decision Processes*, 170, 104149.

doi:10.1016/j.obhdp.2022.104149. Corrigendum (2024), 180, 104278.

43. Kurosu, M., & Kashimura, K. (1995). Apparent usability vs. inherent usability. *CHI '95 Conference Companion*, 292–293. ACM.
44. Tractinsky, N., Katz, A. S., & Ikar, D. (2000). What is beautiful is usable. *Interacting with Computers*, 13(2), 127–145.
45. Hassenzahl, M. (2004). The interplay of beauty, goodness, and usability in interactive products. *Human-Computer Interaction*, 19(4), 319–349.
46. Sonderegger, A., & Sauer, J. (2010). The influence of design aesthetics in usability testing: Effects on user performance and perceived usability. *Applied Ergonomics*, 41(3), 403–410. doi:10.1016/j.apergo.2009.09.002
47. von Restorff, H. (1933). Über die Wirkung von Bereichsbildungen im Spurenfeld. *Psychologische Forschung*, 18, 299–342.
48. Hunt, R. R. (1995). The subtlety of distinctiveness: What von Restorff really did. *Psychonomic Bulletin & Review*, 2(1), 105–112.
49. Wertheimer, M. (1923). Untersuchungen zur Lehre von der Gestalt II. *Psychologische Forschung*, 4, 301–350.
50. Rubin, E. (1915). *Synsoplevede Figurer*. Copenhagen: Gyldendal. (German edition 1921, *Visuell wahrgenommene Figuren*.)
51. Palmer, S. E. (1992). Common region: A new principle of perceptual grouping. *Cognitive Psychology*, 24(3), 436–447.
52. Wagemans, J., Elder, J. H., Kubovy, M., Palmer, S. E., et al. (2012). A century of Gestalt psychology in visual perception: I. *Psychological Bulletin*, 138(6), 1172–1217.
53. Murdock, B. B., Jr. (1962). The serial position effect of free recall. *Journal of Experimental Psychology*, 64(5), 482–488. doi:10.1037/h0045106
54. Glanzer, M., & Cunitz, A. R. (1966). Two storage mechanisms in free recall. *Journal of Verbal Learning and Verbal Behavior*, 5(4), 351–360. doi:10.1016/S0022-5371(66)80044-0
55. Postel, J. (Ed.). (1980, January). *DoD standard Transmission Control Protocol*. RFC 761, section 2.10. rfc-editor.org/rfc/rfc761
56. Thomson, M., & Schinazi, D. (2023). *Maintaining robust protocols*. RFC 9413. doi:10.17487/RFC9413
57. Norman, D. A. (2013). *The Design of Everyday Things* (revised ed.), ch. 5 on slips and mistakes. Basic Books.